Python Basics — Lesson 1

What you'll learn (hands-on)

- Core **Python** syntax: variables, types, lists/dicts, slicing, loops, functions, comprehensions.
- **NumPy** arrays: creation, shapes, broadcasting, vectorized math, boolean masks.
- **Pandas**: `Series`, `DataFrame`, importing/creating data, selection, filtering, grouping, pivoting, missing values.
- **Matplotlib & Seaborn**: quick charts, customization, multi-plots, statistical plots.
- A cohesive **mini case study**: explore and visualize synthetic **bike-share rentals** data.

How to use this notebook

1. Run cells from top to bottom.
2. Try the `# TODO` exercises sprinkled throughout. Most have quick tests.
3. If you're new to Python notebooks: press **Shift+Enter** to run a cell.

In [5]:
```
%pip install numpy pandas matplotlib seaborn openpyxl
%pip install ipywidgets

# Core scientific Python stack
import sys, math, random
import numpy as np #gives the library a name
import pandas as pd

# Plotting
import matplotlib.pyplot as plt
import seaborn as sns

# Plot inline
%matplotlib inline

print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Seaborn:", sns.__version__)
```

```
670.01s - pydevd: Sending message related to process being replaced timed-ou
t after 5 seconds
```

```
Requirement already satisfied: numpy in ./.venv/lib/python3.14/site-packages
(2.5.4)
Requirement already satisfied: pandas in ./.venv/lib/python3.14/site-package
s (3.1.0)
Requirement already satisfied: matplotlib in ./.venv/lib/python3.14/site-pac
kages (3.11.2)
Requirement already satisfied: seaborn in ./.venv/lib/python3.14/site-packag
es (0.13.2)
Requirement already satisfied: openpyxl in ./.venv/lib/python3.14/site-packa
ges (3.1.5)
Requirement already satisfied: python-dateutil>=2.9.0 in ./.venv/lib/python
3.14/site-packages (from pandas) (3.0.0)
Requirement already satisfied: contourpy>=1.0.1 in ./.venv/lib/python3.14/si
te-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in ./.venv/lib/python3.14/site-p
ackages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in ./.venv/lib/python3.14/s
ite-packages (from matplotlib) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in ./.venv/lib/python3.14/s
ite-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in ./.venv/lib/python3.14/sit
e-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in ./.venv/lib/python3.14/site-pack
ages (from matplotlib) (13.0.0)
Requirement already satisfied: pyparsing>=3 in ./.venv/lib/python3.14/site-p
ackages (from matplotlib) (3.3.3)
Requirement already satisfied: et-xmlfile in ./.venv/lib/python3.14/site-pac
kages (from openpyxl) (2.0.0)
Requirement already satisfied: six>=1.6 in ./.venv/lib/python3.14/site-packa
ges (from python-dateutil>=2.9.0->pandas) (1.17.0)
Requirement already satisfied: tzdata in ./.venv/lib/python3.14/site-package
s (from python-dateutil>=2.9.0->pandas) (2026.5)

[notice] A new release of pip is available: 25.3 -> 26.2.2
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
675.92s - pydevd: Sending message related to process being replaced timed-ou
t after 5 seconds
```

```
Requirement already satisfied: ipywidgets in ./.venv/lib/python3.14/site-pac
kages (8.1.9)
Requirement already satisfied: comm>=0.1.3 in ./.venv/lib/python3.14/site-pa
ckages (from ipywidgets) (0.2.3)
Requirement already satisfied: ipython>=6.1.0 in ./.venv/lib/python3.14/site
-packages (from ipywidgets) (9.17.1)
Requirement already satisfied: traitlets>=4.3.1 in ./.venv/lib/python3.14/si
te-packages (from ipywidgets) (5.16.1)
Requirement already satisfied: widgetsnbextension~=4.0.16 in ./.venv/lib/pyt
hon3.14/site-packages (from ipywidgets) (4.0.16)
Requirement already satisfied: jupyterlab_widgets~=3.0.17 in ./.venv/lib/pyt
hon3.14/site-packages (from ipywidgets) (3.0.17)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in ./.venv/li
b/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (1.1.1)
Requirement already satisfied: jedi>=0.18.2 in ./.venv/lib/python3.14/site-p
ackages (from ipython>=6.1.0->ipywidgets) (0.20.1)
Requirement already satisfied: matplotlib-inline>=0.1.6 in ./.venv/lib/pytho
n3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.2.2)
Requirement already satisfied: pexpect>4.6 in ./.venv/lib/python3.14/site-pa
ckages (from ipython>=6.1.0->ipywidgets) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in ./.venv/lib/
python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (3.0.53)
Requirement already satisfied: psutil>=7 in ./.venv/lib/python3.14/site-pack
ages (from ipython>=6.1.0->ipywidgets) (7.2.2)
Requirement already satisfied: pygments>=2.14.0 in ./.venv/lib/python3.14/si
te-packages (from ipython>=6.1.0->ipywidgets) (2.21.0)
Requirement already satisfied: stack_data>=0.6.0 in ./.venv/lib/python3.14/s
ite-packages (from ipython>=6.1.0->ipywidgets) (0.6.3)
Requirement already satisfied: wcwidth>=0.1.4 in ./.venv/lib/python3.14/site
-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0->ipywidgets)
(0.9.4)
Requirement already satisfied: parso<0.9.0,>=0.8.7 in ./.venv/lib/python3.1
4/site-packages (from jedi>=0.18.2->ipython>=6.1.0->ipywidgets) (0.8.7)
Requirement already satisfied: ptyprocess>=0.5 in ./.venv/lib/python3.14/sit
e-packages (from pexpect>4.6->ipython>=6.1.0->ipywidgets) (0.7.0)
Requirement already satisfied: executing>=1.2.0 in ./.venv/lib/python3.14/si
te-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (2.3.0)
Requirement already satisfied: asttokens>=2.1.0 in ./.venv/lib/python3.14/si
te-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (3.0.2)
Requirement already satisfied: pure-eval in ./.venv/lib/python3.14/site-pack
ages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (0.2.4)

[notice] A new release of pip is available: 25.3 -> 26.2.2
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
Python: 3.14.2
NumPy: 2.5.4
Pandas: 3.1.0
Seaborn: 0.13.2
```

1) Python Fundamentals

In [6]:
```
# Variables and basic types
city = "Milano"  #Strings
```

```
bikes = 1200 #Integers
ratio = 7.5 #Floats
is_expanding = True #Booleans
print(type(city), type(bikes), type(ratio), type(is_expanding))
```

```
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
```

In [7]:
```
# Lists and slicing
temps = [12, 15, 13, 18, 21, 20]
```

In [19]:
```
print("first:", temps[0])
print("last two:", temps[-2:])
```

```
first: 12
last two: [21, 20]
```

In [20]:
```
# Dicts
info = {"city": city, "bikes": bikes, "ratio": ratio}
```

In [21]:
```
info["country"] = "Italy"
```

In [22]:
```
print(info)
```

```
{'city': 'Milano', 'bikes': 1200, 'ratio': 7.5, 'country': 'Italy'}
```

In [23]:
```
# Control flow
labels = []
for t in temps:
    if t >= 18:
        labels.append("warm")
    else:
        labels.append("cool")
print(labels)
```

```
['cool', 'cool', 'cool', 'warm', 'warm', 'warm']
```

In [24]:
```
labels = []
i = 0
while i < len(temps):
    t = temps[i]
    if t >= 18:
        labels.append("warm")
    else:
        labels.append("cool")
    i += 1
print(labels)
```

```
['cool', 'cool', 'cool', 'warm', 'warm', 'warm']
```

In [25]:
```
# List comprehension (equivalent)
labels2 = ["warm" if t>=18 else "cool" for t in temps]
print(labels2)
```

```
['cool', 'cool', 'cool', 'warm', 'warm', 'warm']
```

In [26]:
```
# Functions
def fahrenheit(c):
```

```
    return 32 + 9/5 * c
```

In [27]:
```
fahrenheit(20)
```

Out[27]: 68.0

In [28]:
```
print([fahrenheit(t) for t in temps])
```

```
[53.6, 59.0, 55.400000000000006, 64.4, 69.80000000000001, 68.0]
```

Mini exercise

Task: Write a function `safe_div(a, b)` that returns `a/b`, but if `b==0` return `None`.

In [30]:
```
# TODO: implement safe_div
def safe_div(a, b):
#    if b == 0:
#        return None
#    else:
#        return a/b
    return a/b if b!=0 else None

# Quick tests
assert safe_div(6,2) == 3
assert safe_div(5,0) is None
print("All tests passed!")
```

```
All tests passed!
```

2) NumPy basics

fast arrays and vectorization

In [ ]:
```
arr = np.array([1,2,3,4,5]) # only column array. input is an list
print("shape:", arr.shape, "dtype:", arr.dtype)
```

```
shape: (5,) dtype: int64
```

In [32]:
```
# Vectorized operations
print("arr * 10:", arr * 10)
print("exp(arr):", np.exp(arr))
```

```
arr * 10: [10 20 30 40 50]
exp(arr): [  2.71828183   7.3890561   20.08553692  54.59815003 148.4131591 ]
```

In [ ]:
```
matrix = np.array([[1,2,3],[4,5,6]])
print("matrix shape:", matrix.shape)
print("matrix sum:", matrix.sum())
print("matrix sum axis=0:", matrix.sum(axis=0))
# scalar product: * or @
```

```
matrix shape: (2, 3)
matrix sum: 21
matrix sum axis=0: [5 7 9]
```

In [34]:
```
matrix[0,0]
```

Out[34]:
```
np.int64(1)
```

In [35]:
```
A = np.random.randint(0, 10, (3,4))
```

In [36]:
```
A
```

Out[36]:
```
array([[6, 9, 9, 8],
       [3, 7, 9, 9],
       [8, 8, 2, 5]])
```

In [ ]:
```
# Boolean masks
mask = A % 2 == 0
print("even mask:\n", mask)
print("A[mask]\n", A[mask]) # only avlule with true
```

```
even mask:
 [[ True False False  True]
 [False False False False]
 [ True  True  True False]]
A[mask]
 [6 8 8 8 2]
```

Mini exercise (NumPy)

Create a function `normalize(v)` that returns `v` scaled to the range `[0,1]` (min-max scaling).

In [39]:
```
# TODO: implement normalize(v)
def normalize(v: np.ndarray) -> np.ndarray:
    v = np.asarray(v, dtype=float)
    low, high = v.min(), v.max()
    v = (v - low) / (high - low) if high > low else np.zeros_like(v)
    return v

# Test
x = np.array([2.0, 4.0, 6.0])
nx = normalize(x)
assert np.isclose(nx.min(), 0.0) and np.isclose(nx.max(), 1.0)
print("NumPy test passed!")
```

```
NumPy test passed!
```

3) Pandas - tabular data power tools

Case Study: Bike-Share Rentals You're analyzing a city's bike-share program. You'll explore **daily usage** and how it's affected by **temperature**, **weather**, and

marketing.

365 days of data for a single year:

- `date` : daily timestamps
- `temp_c` : temperature in °C
- `is_rain` : 0/1 rain flag
- `weekday` : Mon-Sun
- `holiday` : 0/1
- `marketing` : budget level (Low/Med/High)
- `rentals` : number of bikes rented that day

Let's load and preview the dataset.

In [40]:
```
df = pd.read_excel('bike_share_data.xlsx')
```

In [41]:
```
df.head()
```

Out[41]:

	date	temp_c	is_rain	weekday	holiday	marketing	rentals	weekday_nan
0	2024-01-01	-0.9	0	0	0	Med	632	Mond
1	2024-01-02	-4.9	0	1	0	Med	495	Tuesd
2	2024-01-03	0.6	1	2	0	Low	484	Wednesd
3	2024-01-04	1.2	0	3	0	Med	736	Thursd
4	2024-01-05	-7.4	0	4	0	Med	559	Frid

In [42]:
```
# Basic selection
df.shape
```

Out[42]:
```
(365, 8)
```

In [43]:
```
df.columns.tolist()
```

Out[43]:
```
['date',
 'temp_c',
 'is_rain',
 'weekday',
 'holiday',
 'marketing',
 'rentals',
 'weekday_name']
```

In [44]:
```
df.dtypes
```

Out[44]:
```
date            datetime64[us]
temp_c                 float64
is_rain                  int64
weekday                  int64
holiday                  int64
marketing                  str
rentals                  int64
weekday_name               str
dtype: object
```

In [45]:
```
df[df['temp_c']>20]
```

Out[45]:

	date	temp_c	is_rain	weekday	holiday	marketing	rentals	weekday_n
118	2024-04-28	22.1	1	6	0	Med	948	Su
123	2024-05-03	22.2	1	4	0	High	1010	F
124	2024-05-04	20.7	0	5	0	High	1100	Satu
126	2024-05-06	22.9	1	0	0	High	1116	Mo
132	2024-05-12	21.3	1	6	0	High	991	Su
136	2024-05-16	20.8	0	3	0	High	1110	Thur
137	2024-05-17	22.5	0	4	0	High	1153	F
138	2024-05-18	26.1	1	5	0	High	1062	Satu
139	2024-05-19	28.9	0	6	0	High	1291	Su
140	2024-05-20	21.5	1	0	0	High	1025	Mo
143	2024-05-23	21.4	0	3	0	High	1147	Thur
147	2024-05-27	20.5	1	0	1	High	824	Mo
148	2024-05-28	24.2	1	1	0	High	1124	Tue
149	2024-05-29	21.6	1	2	0	High	1081	Wedne
150	2024-05-30	20.7	0	3	0	High	1065	Thur
154	2024-06-03	21.3	1	0	0	High	968	Mo
155	2024-06-04	26.8	0	1	1	High	1238	Tue
156	2024-06-05	22.0	0	2	0	High	1143	Wedne
157	2024-06-06	24.6	0	3	0	High	1243	Thur
158	2024-06-07	20.2	0	4	0	High	1236	F
162	2024-06-11	28.2	0	1	0	High	1357	Tue

	date	temp_c	is_rain	weekday	holiday	marketing	rentals	weekday_n
164	2024-06-13	24.4	1	3	0	High	1087	Thur
166	2024-06-15	24.7	0	5	0	High	1179	Satu
167	2024-06-16	23.1	0	6	0	High	1152	Su
168	2024-06-17	21.5	0	0	0	High	1019	Mo
169	2024-06-18	21.9	1	1	0	High	996	Tue
171	2024-06-20	23.3	0	3	0	High	1150	Thur
172	2024-06-21	20.6	0	4	0	High	895	F
175	2024-06-24	22.5	0	0	0	High	1116	Mo
176	2024-06-25	26.7	0	1	0	High	1282	Tue
177	2024-06-26	22.4	0	2	0	High	1056	Wedne
178	2024-06-27	21.6	0	3	0	High	1267	Thur
179	2024-06-28	22.8	0	4	0	High	1164	F
180	2024-06-29	25.8	0	5	1	High	1299	Satu
181	2024-06-30	22.5	0	6	0	High	1396	Su
182	2024-07-01	20.6	0	0	0	High	1253	Mo
183	2024-07-02	25.1	1	1	0	High	1017	Tue
184	2024-07-03	23.0	0	2	0	High	1132	Wedne
185	2024-07-04	26.3	0	3	0	High	1219	Thur
186	2024-07-05	22.2	1	4	0	High	1024	F
189	2024-07-08	26.4	0	0	0	High	1365	Mo
190	2024-07-09	26.6	0	1	0	High	1242	Tue

	date	temp_c	is_rain	weekday	holiday	marketing	rentals	weekday_n
191	2024-07-10	20.8	0	2	0	High	1319	Wedne
192	2024-07-11	20.1	1	3	0	High	949	Thur
193	2024-07-12	25.6	0	4	0	High	1374	F
196	2024-07-15	22.9	0	0	0	High	1120	Mo
198	2024-07-17	20.7	0	2	0	High	1170	Wedne
199	2024-07-18	20.2	1	3	1	High	1023	Thur
200	2024-07-19	21.6	0	4	0	High	1155	F
201	2024-07-20	24.7	0	5	0	High	1151	Satu
202	2024-07-21	20.6	0	6	0	High	1063	Su
203	2024-07-22	22.2	0	0	0	High	1049	Mo
210	2024-07-29	22.2	0	0	0	High	1150	Mo
215	2024-08-03	21.8	0	5	0	High	1231	Satu
216	2024-08-04	20.2	0	6	0	High	1209	Su
217	2024-08-05	22.5	0	0	0	High	1179	Mo
226	2024-08-14	24.6	1	2	0	High	1030	Wedne
227	2024-08-15	22.5	1	3	0	High	940	Thur
244	2024-09-01	20.1	0	6	0	Med	1112	Su
248	2024-09-05	21.7	0	3	0	Low	874	Thur

In [46]:
```
df[(df['temp_c']>20) & (df['is_rain']==0)]
```

Out[46]:

	date	temp_c	is_rain	weekday	holiday	marketing	rentals	weekday_n
124	2024-05-04	20.7	0	5	0	High	1100	Satu
136	2024-05-16	20.8	0	3	0	High	1110	Thur
137	2024-05-17	22.5	0	4	0	High	1153	F
139	2024-05-19	28.9	0	6	0	High	1291	Su
143	2024-05-23	21.4	0	3	0	High	1147	Thur
150	2024-05-30	20.7	0	3	0	High	1065	Thur
155	2024-06-04	26.8	0	1	1	High	1238	Tue
156	2024-06-05	22.0	0	2	0	High	1143	Wedne
157	2024-06-06	24.6	0	3	0	High	1243	Thur
158	2024-06-07	20.2	0	4	0	High	1236	F
162	2024-06-11	28.2	0	1	0	High	1357	Tue
166	2024-06-15	24.7	0	5	0	High	1179	Satu
167	2024-06-16	23.1	0	6	0	High	1152	Su
168	2024-06-17	21.5	0	0	0	High	1019	Mo
171	2024-06-20	23.3	0	3	0	High	1150	Thur
172	2024-06-21	20.6	0	4	0	High	895	F
175	2024-06-24	22.5	0	0	0	High	1116	Mo
176	2024-06-25	26.7	0	1	0	High	1282	Tue
177	2024-06-26	22.4	0	2	0	High	1056	Wedne
178	2024-06-27	21.6	0	3	0	High	1267	Thur
179	2024-06-28	22.8	0	4	0	High	1164	F

	date	temp_c	is_rain	weekday	holiday	marketing	rentals	weekday_n
180	2024-06-29	25.8	0	5	1	High	1299	Satu
181	2024-06-30	22.5	0	6	0	High	1396	Su
182	2024-07-01	20.6	0	0	0	High	1253	Mo
184	2024-07-03	23.0	0	2	0	High	1132	Wedne
185	2024-07-04	26.3	0	3	0	High	1219	Thur
189	2024-07-08	26.4	0	0	0	High	1365	Mo
190	2024-07-09	26.6	0	1	0	High	1242	Tue
191	2024-07-10	20.8	0	2	0	High	1319	Wedne
193	2024-07-12	25.6	0	4	0	High	1374	F
196	2024-07-15	22.9	0	0	0	High	1120	Mo
198	2024-07-17	20.7	0	2	0	High	1170	Wedne
200	2024-07-19	21.6	0	4	0	High	1155	F
201	2024-07-20	24.7	0	5	0	High	1151	Satu
202	2024-07-21	20.6	0	6	0	High	1063	Su
203	2024-07-22	22.2	0	0	0	High	1049	Mo
210	2024-07-29	22.2	0	0	0	High	1150	Mo
215	2024-08-03	21.8	0	5	0	High	1231	Satu
216	2024-08-04	20.2	0	6	0	High	1209	Su
217	2024-08-05	22.5	0	0	0	High	1179	Mo
244	2024-09-01	20.1	0	6	0	Med	1112	Su
248	2024-09-05	21.7	0	3	0	Low	874	Thur

In [47]:
```
# Filtering & sorting
subset = df[(df['temp_c']>20) & (df['is_rain']==0)].sort_values('rentals', a
subset
```

Out[47]:

	date	temp_c	is_rain	weekday	holiday	marketing	rentals	weekday_n
181	2024-06-30	22.5	0	6	0	High	1396	Su
193	2024-07-12	25.6	0	4	0	High	1374	F
189	2024-07-08	26.4	0	0	0	High	1365	Mo
162	2024-06-11	28.2	0	1	0	High	1357	Tue
191	2024-07-10	20.8	0	2	0	High	1319	Wedne

In [48]:
```
df.groupby('weekday_name')['rentals'].mean()
```

Out[48]:
```
weekday_name
Friday       802.788462
Monday       816.867925
Saturday     851.557692
Sunday       845.519231
Thursday     797.038462
Tuesday      816.865385
Wednesday    806.019231
Name: rentals, dtype: float64
```

In [49]:
```
# Groupby: average rentals by weekday
avg_by_weekday = df.groupby('weekday_name')['rentals'].mean().sort_values(as
avg_by_weekday
```

Out[49]:
```
weekday_name
Saturday     851.557692
Sunday       845.519231
Monday       816.867925
Tuesday      816.865385
Wednesday    806.019231
Friday       802.788462
Thursday     797.038462
Name: rentals, dtype: float64
```

In [50]:
```
# Pivot: average rentals by marketing and rain
pivot_tbl = pd.pivot_table(df, values='rentals', index='marketing', columns=
pivot_tbl
```

Out[50]:

is_rain	0	1
marketing		
High	1109.397590	956.750000
Low	676.645455	533.277778
Med	794.512821	632.527778

Mini exercise (Pandas)

Compute the **median** `rentals` per `marketing` level and store it as `median_by_marketing`.

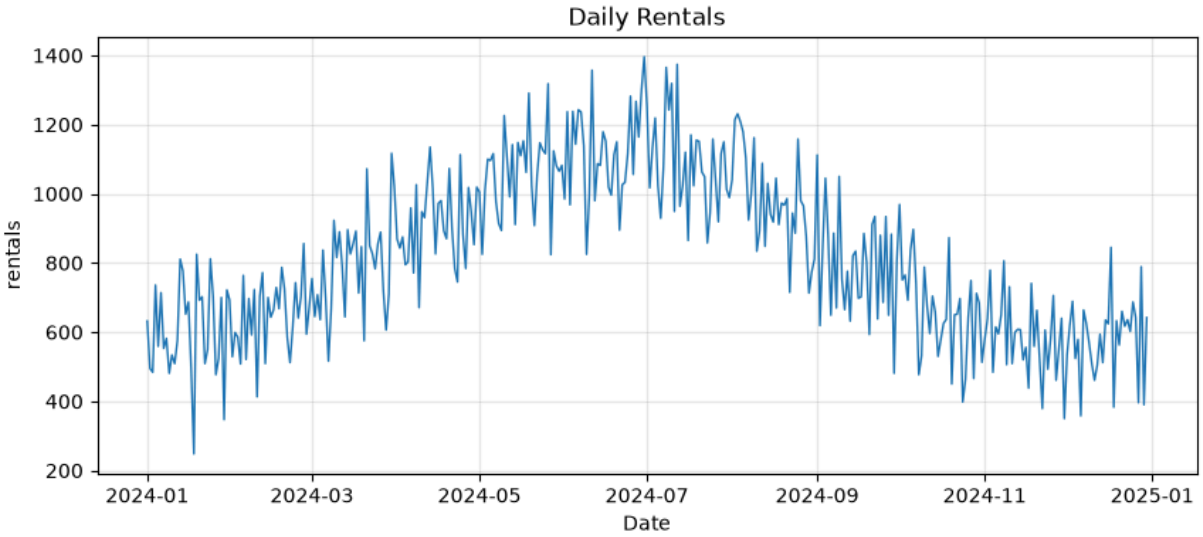
In [52]:
```
# TODO: compute median_by_marketing
median_by_marketing = df.groupby("marketing")["rentals"].median()

# Test (basic checks)
assert median_by_marketing is not None
assert set(median_by_marketing.index).issubset({'Low','Med','High'})
print("Pandas test passed!")
```

```
Pandas test passed!
```

4) Matplotlib — quick charts & customization

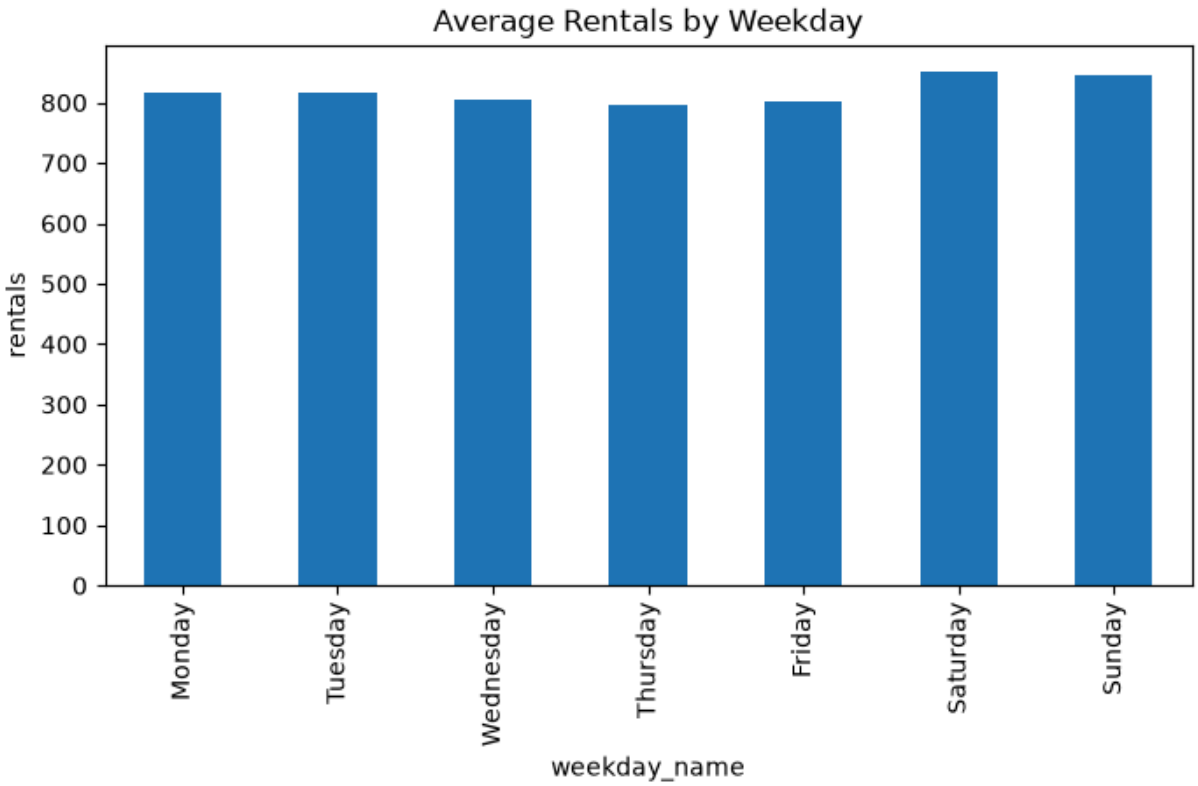
In [53]:
```
plt.figure(figsize=(10,4)) # (width, height)
plt.plot(df['date'], df['rentals'], linewidth=1) # x, y
plt.title('Daily Rentals') # title
plt.xlabel('Date'); plt.ylabel('rentals') # axis labels
plt.grid(True, alpha=0.3) # grid
plt.show() # show the plot
```



In [54]:
```
df.groupby('weekday_name')['rentals'].mean()
```

Out[54]:
```
weekday_name
Friday       802.788462
Monday       816.867925
Saturday     851.557692
Sunday       845.519231
Thursday     797.038462
Tuesday      816.865385
Wednesday    806.019231
Name: rentals, dtype: float64
```

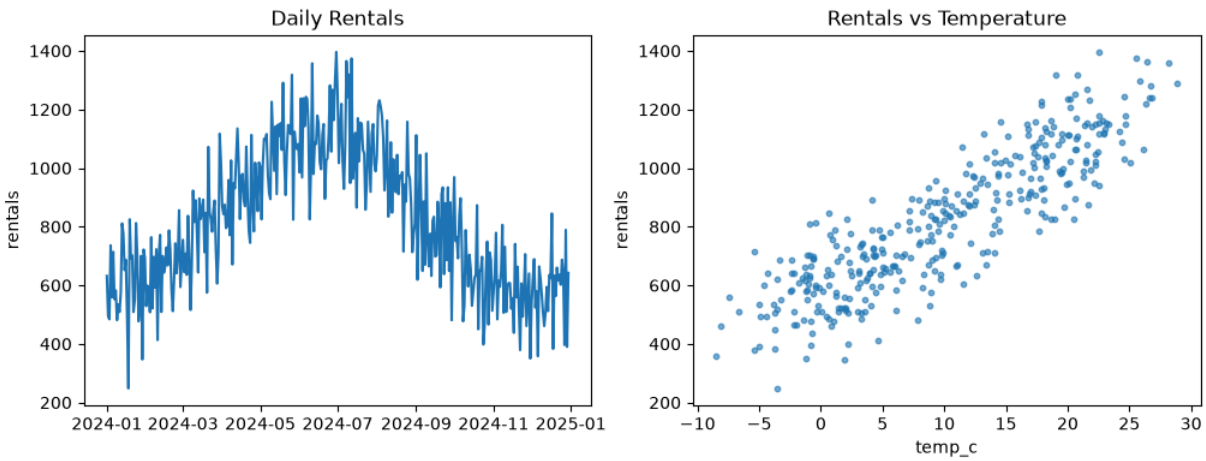
In [55]:
```
# Bar chart from a groupby
avg_by_day = df.groupby('weekday_name')['rentals'].mean().reindex(
    ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
)
plt.figure(figsize=(8,4))
avg_by_day.plot(kind='bar')
plt.title('Average Rentals by Weekday')
plt.ylabel('rentals')
plt.show()
```



In [56]:
```
fig, ax = plt.subplots(1,2, figsize=(12,4)) #Two subplots side by side, 1 ro
ax[0].plot(df['date'], df['rentals'])
ax[0].set_title('Daily Rentals')
ax[0].set_ylabel('rentals')

ax[1].scatter(df['temp_c'], df['rentals'], s=10, alpha=0.6)
ax[1].set_title('Rentals vs Temperature')
ax[1].set_xlabel('temp_c')
```

```
ax[1].set_ylabel('rentals')
plt.show()
```



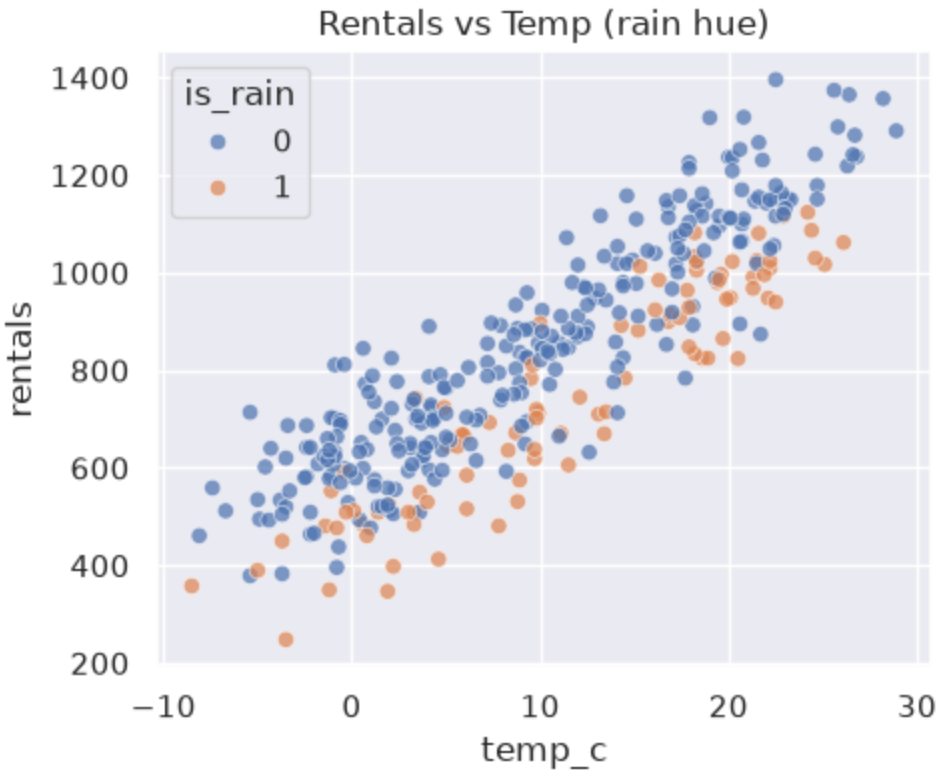
In [57]:
```
# Interactivity
try:
    from ipywidgets import interact, interactive, widgets, fixed
    _WIDGETS_OK = True
except Exception as e:
    print("ipywidgets not available; interactive parts will be minimal. Erro
    _WIDGETS_OK = False


if _WIDGETS_OK:
    @interact(min_temp=(-5,35,1), max_temp=(0,40,1), rain=widgets.Dropdown(o
    def filter_plot(min_temp=10, max_temp=30, rain='All'):
        q = (df['temp_c']>=min_temp) & (df['temp_c']<=max_temp)
        if rain == 'No rain':
            q &= (df['is_rain']==0)
        elif rain == 'Rain':
            q &= (df['is_rain']==1)
        dff = df[q]
        plt.figure(figsize=(6,4))
        sns.scatterplot(data=dff, x='temp_c', y='rentals', hue='marketing',
        plt.title(f'Scatter (n={len(dff)})')
        plt.show()
else:
    print("Widgets unavailable; skipping interactive demo.")
```

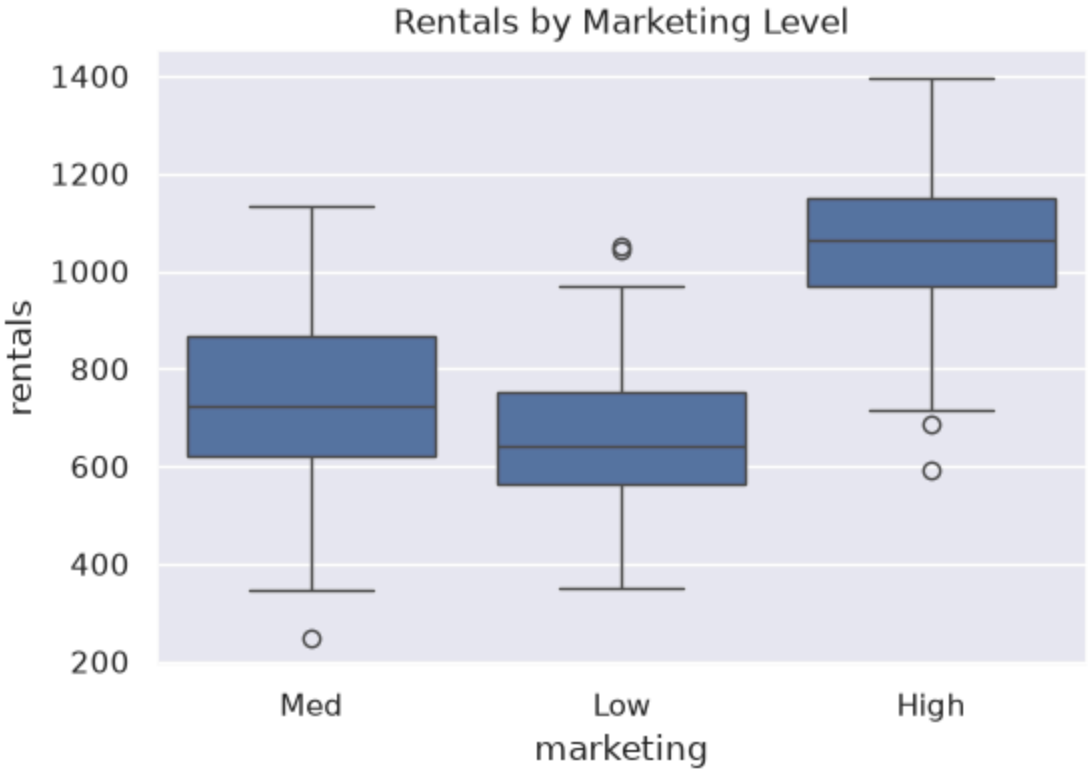
```
interactive(children=(IntSlider(value=10, description='min_temp', max=35, mi
n=-5), IntSlider(value=30, descrip…
```

5) Seaborn — beautiful statistical plots

In [58]:
```
sns.set_theme() # set seaborn theme
plt.figure(figsize=(5,4))
sns.scatterplot(data=df, x='temp_c', y='rentals', hue='is_rain', alpha=0.7)
plt.title('Rentals vs Temp (rain hue)')
plt.show()
```

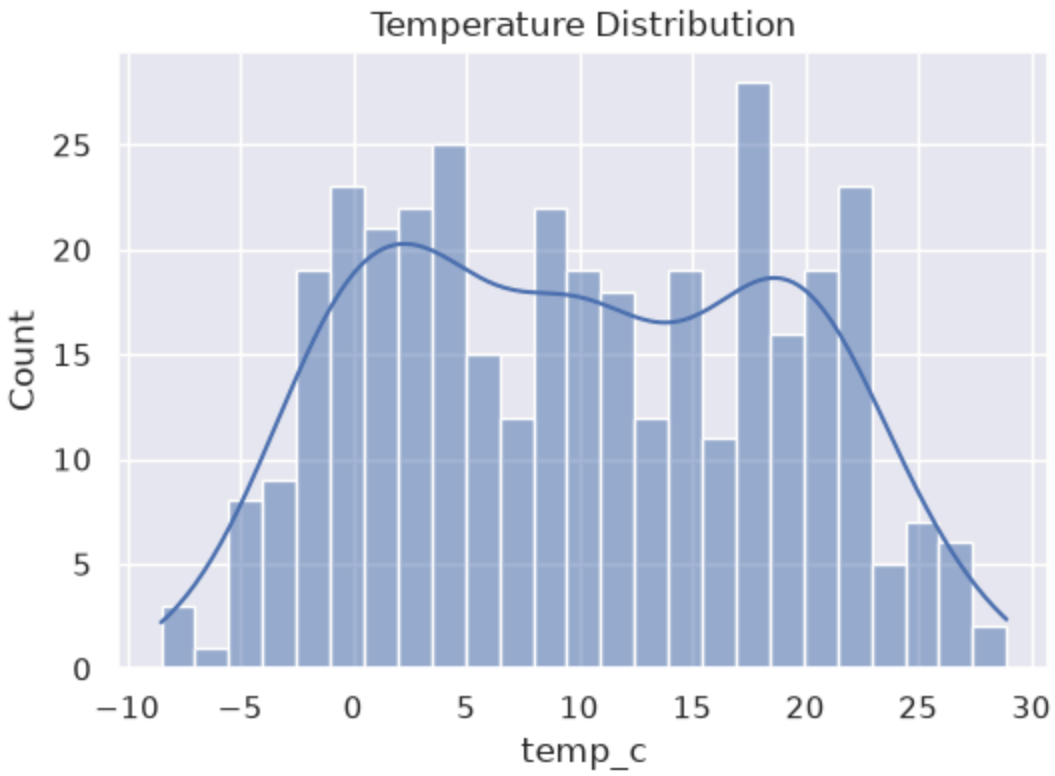


In [59]:
```
plt.figure(figsize=(6,4))
sns.boxplot(data=df, x='marketing', y='rentals')
plt.title('Rentals by Marketing Level')
plt.show()
```

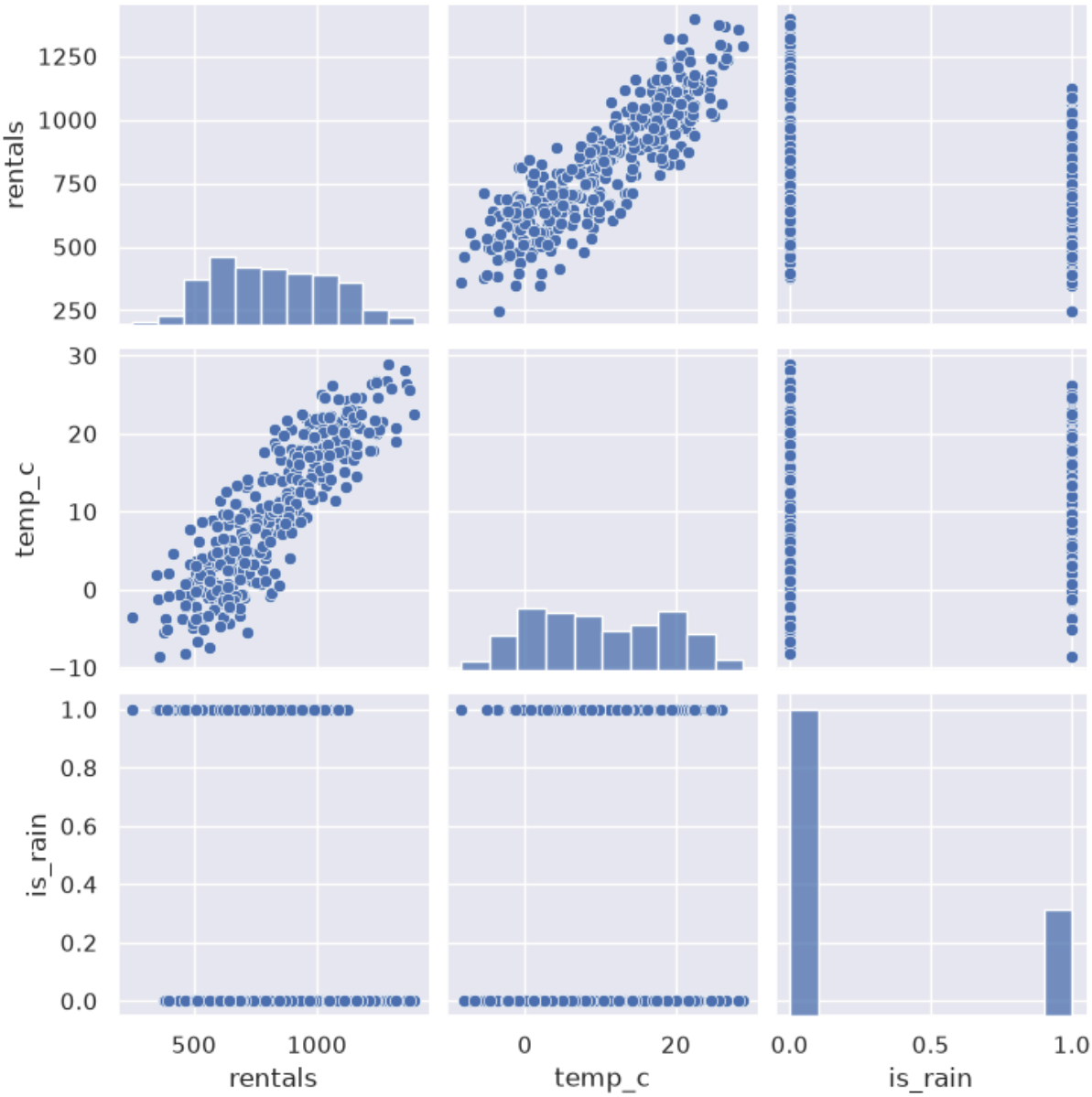


In [60]:
```
plt.figure(figsize=(6,4))
sns.histplot(data=df, x='temp_c', bins=25, kde=True)
```

```
plt.title('Temperature Distribution')
plt.show()
```



In [61]:
```
# Pairwise relationships
sns.pairplot(df[['rentals','temp_c','is_rain']])
plt.show()
```



6) Mini Analysis Challenge (10–15 min)

Answer these **three** questions using the tools above:

1. Which **weekday** tends to have the **highest average rentals**?
2. Does **rain** reduce rentals? Quantify the average difference on rainy vs non-rainy days.
3. Which **marketing** level appears most effective, controlling roughly for temperature (e.g., filter `temp_c` between 15 and 25)?

> Put your code in the next cell. There's a lightweight check after it.

In [64]:
```
highest_weekday = df.groupby("weekday_name") ['rentals' ] .mean() .sort_valu

rainy_vs_not = df.groupby('is_rain')["rentals"].mean().to_dict()
```

```
mid = df[ (df['temp_c']>=15) & (df['temp_c'] <= 25) ]
best_marketing = mid.groupby('marketing') ['rentals' ].mean().sort_values(as

print(highest_weekday, rainy_vs_not, best_marketing)
```

```
Saturday {0: 843.1107011070111, 1: 751.4893617021277} High
```

In [63]:
```
# Lightweight checks (they won't reveal the exact answer, but guard basic lo
assert highest_weekday is not None
assert rainy_vs_not is not None
assert best_marketing_midtemp is not None
print("Great! You produced three results — discuss them with your neighbor □
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[63], line 2
      1 # Lightweight checks (they won't reveal the exact answer, but guard
basic logic)
----> 2 assert highest_weekday is not None
      3 assert rainy_vs_not is not None
      4 assert best_marketing_midtemp is not None
      5 print("Great! You produced three results — discuss them with your ne
ighbor □")

AssertionError:
```

Wrap-up

Basic introduction to Python, learn more about the following libraries (numpy, matplotlib, seaborn, pandas) that will be the basis of every Python project

Keyboard cheat-sheet

- Run cell: **Shift+Enter**
- Insert cell below: **B**
- Interrupt kernel: **I, I**
- Show command palette: **Ctrl/Cmd + Shift + P**

(Optional) Solutions — peek only after trying!

In [ ]:
```
# Solutions to earlier TODOs

def safe_div(a, b):
    return a/b if b != 0 else None
```

```
def normalize(v: np.ndarray) -> np.ndarray:
    v = np.asarray(v, dtype=float)
    lo, hi = v.min(), v.max()
    return (v - lo) / (hi - lo) if hi > lo else np.zeros_like(v)

median_by_marketing = df.groupby('marketing')['rentals'].median()

# Mini analysis exemplar (your answers may vary a bit by rounding/filter cho
highest_weekday = df.groupby('weekday_name')['rentals'].mean().sort_values(a
rainy_vs_not = df.groupby('is_rain')['rentals'].mean().to_dict()
mid = df[(df['temp_c']>=15)&(df['temp_c']<=25)]
best_marketing_midtemp = mid.groupby('marketing')['rentals'].mean().sort_val

highest_weekday, rainy_vs_not, best_marketing_midtemp
```

In [ ]:
```
df.groupby('is_rain')['rentals'].mean().to_dict()
```

In [ ]:
```

```